Rapid estimation of global sea surface temperatures from sparse streaming in situ observations

Cassidy All[1], Kevin Ho[2], Maya Magnuski[3], Christopher Nicolaides[4], Louisa B. Ebby[5], Mohammad Farazmand[5]

[1]Department of Applied Mathematics, University of Colorado Boulder, 11 Engineering Drive, Boulder, CO 80309
[2]Department of Mathematics and Statistics, Mississippi State University, 175 President's Circle, MS 39762
[3]Mathematics Program, Bard College, 30 Campus Road, Annandale-On-Hudson, NY 12504
[4]Department of Mathematics, Indiana University, 831 East 3rd Street, Bloomington, IN 47405
[5]Department of Mathematics, North Carolina State University, 2311 Stinson Drive, Raleigh, NC 27695

Key Points:

- We introduce S-DEIM as a new method for accurate reconstruction of spatiotemporal geophysical quantities from extremely sparse observations.
- S-DEIM reconstructions are achieved through a synergistic and rigorous combination of empirical interpolation and deep learning.
- By applying it to SST reconstruction, we demonstrate that S-DEIM is robust to observational noise and sensor placement method.

Corresponding author: M. Farazmand, `farazmand@ncsu.edu`

Abstract

Reconstructing high-resolution sea surface temperatures (SST) from staggered SST measurements is essential for analyzing earth system processes. However, when SST measurements are sparse, the resulting inferred SST fields are rather inaccurate. Here, we introduce Sparse Discrete Empirical Interpolation Method (S-DEIM) as a model-free data assimilation method to reconstruct high-resolution SST fields from sparse in situ observations. The S-DEIM estimate consists of two terms, one computed from instantaneous in situ observations using empirical interpolation, and the other learned from the historical time series of observations using recurrent neural networks (RNNs). We train the RNNs using the National Oceanic and Atmospheric Administration's weekly high-resolution SST dataset spanning the years 1989-2021 which constitutes the training data. Subsequently, we examine the performance of S-DEIM on the test data, comprising January 2022 to January 2023. For this test data, S-DEIM infers the high-resolution SST from 100 in situ observations, constituting only 0.2% of the high-resolution spatial grid. We show that the resulting S-DEIM reconstructions are about 40% more accurate than earlier empirical interpolation methods, such as DEIM and Q-DEIM. Furthermore, 91% of S-DEIM estimates fall within $\pm 1^\circ$C of the true SST. We also demonstrate that S-DEIM is robust with respect to sensor placement: even when the sensors are distributed randomly, S-DEIM reconstruction error deteriorates only by 1-2%. S-DEIM is also computationally efficient: training the RNN, which is performed only once offline, takes approximately one minute. Once trained, the S-DEIM reconstructions are computed in less than a second. As such, S-DEIM can be used for rapid SST reconstruction from sparse streaming observational data in real time. Although here we focus on SST, the same S-DEIM framework can be used for reconstructing other geophysical quantities from sparse measurements.

Plain Language Summary

Monitoring, forecasting, and analyzing ocean-atmosphere processes rely on high-resolution geophysical quantities, such as temperature, wind, and humidity. When observational data is sparse, estimating these spatiotemporal quantities becomes imprecise. In this study, we investigate the ability of the Sparse Discrete Empirical Interpolation Method (S-DEIM)—a data-driven, model-free method—to infer high-resolution quantities from limited observations. S-DEIM integrates conventional empirical interpolation with modern deep learning techniques through a mathematically rigorous framework. We demonstrate its efficacy by reconstructing global sea surface temperatures, achieving results in seconds with a 40% improvement in accuracy over conventional interpolation methods.

1 Introduction

Sea surface temperature (SST) is a critical piece of information for understanding various meteorological and oceanographic processes. For example, short-term weather forecasts (Chelton & Wentz, 2005), seasonal to long-term climate predictions (Banzon et al., 2016; Goddard et al., 2001), and understanding marine ecosystems (Venegas et al., 2023) rely on sea surface temperature measurements.

SST observations are gathered by two predominant modalities: remote sensing through satellites and direct in situ observations through buoys, drifters, and ships. In situ measurements provide highly accurate local SST measurements but are limited in number due to logistical and economic constraints. Although satellite-based measurements via infrared and microwave radiometers offer broader spatial coverage, cloud cover and atmospheric conditions can interfere with the accuracy of their observations (Koner et al., 2016). Therefore, SST measurements alone do not provide accurate and high-fidelity SST fields. Instead, as we review in Section 1.1, these measurements are used in a data assimilation process to obtain accurate and high-resolution estimates of the SST field.

The main objective of the present paper is to investigate the efficacy of a new data assimilation method, namely the Sparse Discrete Empirical Interpolation Method (S-DEIM) (Farazmand, 2024), for estimating high-resolution SST fields from very sparse in situ observations. S-DEIM consists of two components. One component is determined from instantaneous in situ observations, using classical interpolation techniques. The other component is estimated from time series of past in situ observations, using recurrent neural networks (RNNs) (Farazmand, 2026).

The resulting RNN-based S-DEIM method is model-free, i.e., it does not require a geophysical fluid dynamics model. Furthermore, it reproduces reasonably accurate SST estimates even when in situ observations are very sparse. Finally, the estimates can be reproduced rapidly, within a few seconds, rendering it particularly attractive for real-time nowcasting of SST fields. The S-DEIM estimates are not as accurate as model-assisted data assimilation methods, such as variational methods and ensemble Kalman filtering (see Section 1.1 for a review). However, the S-DEIM estimate provides an excellent initial guess for such model-assisted data assimilation techniques.

1.1 Background and Related Work

In this section, we review different types of available observational data and data assimilation methods that are common in physical oceanography. We also discuss previous studies that use DEIM-based methods in the context of SST reconstruction.

Observational data: SST observations are collected primarily through two methods: (i) remote sensing and (ii) direct in situ measurements (Kent et al., 2010). Remote sensing is performed through satellites that carry infrared and microwave sensors (Dickey et al., 2006). Although satellites offer broad spatial coverage, they measure radiance, which can be affected by atmospheric conditions, leading to high measurement uncertainty. Additionally, infrared sensors cannot make observations through clouds, contributing to gaps in satellite coverage (Minnett et al., 2019). On the other hand, in situ measurements, collected directly using moored buoys, ships, and surface drifters, tend to be more accurate (Kennedy, 2013). However, these in situ observations are limited by their relatively sparse spatial distribution.

Reanalysis: In oceanography, the term "reanalysis" refers to the blending of high-fidelity geophysical models and observational measurements through data assimilation to improve the accuracy of SST estimation and fill the gaps where observational data is lacking. The choice of the data assimilation method must strike a balance between computational cost and the desired accuracy. For example, variational data assimilation methods (Courtier et al., 1998, 1994; Cummings & Smedstad, 2013) are the most accurate, but demand substantial computational resources, since they require several forward solves of the geophysical model and backward-in-time solves of the model adjoint. A computationally less demanding method is the ensemble Kalman filter, which requires repeated solves of the forward model, but does not require the adjoint equations (Evensen, 2003; Rozier et al., 2007; Gorthi et al., 2011). The National Atmospheric and Oceanic Administration (NOAA) uses Optimum Interpolation (OI) to create high-resolution SST fields, producing weekly and daily datasets (Reynolds et al., 2002). OI is similar to Kalman filtering, but it simplifies the data assimilation process by assuming a static covariance matrix to reduce computational cost (Reynolds & Smith, 1994; Reynolds et al., 2007).

Model-free SST reconstruction: In the past decade, with the advent of deep learning and artificial intelligence, there has been a growing interest in purely data-driven reconstruction and forecast of geophysical quantities, such as SST (Bi et al., 2023; Bodnar et al., 2025; Pathak et al., 2022). These model-free methods use the abundant reanalysis data, going back several decades, to train deep learning models. Once trained, the deep learning model is in turn used for rapid SST reconstruction and forecast (Taylor & Feng, 2022; Xu et al., 2023). A promising deep learning tool is the so-called autoencoders which first learn a mapping from high-fidelity reanalysis data to sparse observations. Then a second mapping is learned from sparse observations back to the high-fidelity data. The two maps are constrained so that their composition approximates the identity mapping. Although autoencoders have had some success in reconstruction and forecast of regional SST (Barth et al., 2022; Goh et al., 2024; Lobashev et al., 2023), they are not yet adept at reconstructing global SST from sparse observations. For example, as we discuss in Appendix B, a deep convolutional autoencoder failed to outperform our S-DEIM reconstructions.

DEIM and Q-DEIM: DEIM is a particular example of a model-free method for SST reconstruction, which is notable for its interpretability and low computational cost. Although originally developed for reduced-order modeling (Barrault et al., 2004; Chaturantabut & Sorensen, 2010), it was later repurposed for flow reconstruction from incomplete observations (Manohar et al., 2018). As an empirical interpolation, DEIM partially owes its success to the empirical basis functions (or modes) in which the interpolant is expanded. Instead of using a prescribed basis, such as polynomials, DEIM uses an empirical basis which is derived specifically for the system of interest. In the case of SST reconstructions, the basis is derived by performing Proper Orthogonal Decomposition (POD) on historical reanalysis data. In this regard, DEIM is similar to the Empirical Orthogonal Functions (EOFs) method (Beckers & Rixen, 2003; Smith et al., 1996). The expansion coefficients within the POD basis are obtained by solving a linear least squares problem (see Section 3).

The quality of DEIM reconstructions are highly sensitive to the locations of the sensor measurements. Drmač and Gugercin (2016) introduced the Q-DEIM method, which uses column-pivoted QR (CPQR) factorization to approximate optimal sensor locations. Alternative sensor placement methods in the context of SST reconstruction have also been explored. For example, Saito et al. (2021) developed a determinant-based sensor placement method that seeks to minimize uncertainties in the reconstruction. Clark et al. (2021) consider the multi-fidelity sensor placement problem with cost constraints, introducing greedy algorithms for placing a mix of sensors with varying accuracy. Klishin et al. (2023) introduce a sensor placement method based on ideas from statistical physics. We refer to Karnik et al. (2025) for a comprehensive review of sensor placement methods.

Another aspect of DEIM which has received much attention is its sensitivity to observational noise (Argaud et al., 2017; Peherstorfer et al., 2020). In particular, Callaham et al. (2019) investigate a variety of sparsity-promoting optimization problems instead of linear least squares. They find that sparse representations improve the robustness of the reconstructions to observational noise.

S-DEIM: DEIM was originally developed under the assumption that the number of sensors r is equal to the number of modes m (Chaturantabut & Sorensen, 2010). The overdetermined regime, where $r > m$, has also received much attention. In fact, when $r > m$, DEIM coincides with the gappy POD method (Everson & Sirovich, 1995), which predates it by more than a decade. Numerical evidence suggests that DEIM estimates become more accurate as the number of sensor measurements increase for a fixed number of modes (Clark et al., 2021). In the overdetermined regime ($r > m$), there is also evidence that the reconstructions are more robust to observational noise (Argaud et al., 2017; Peherstorfer et al., 2020).

In contrast, the underdetermined regime ($r < m$), where fewer sensor measurements are available, has only recently attracted attention (Farazmand, 2024). This regime can occur when the number of sensor measurements r are very limited, but $m = r$ modes do not provide enough model complexity to faithfully reconstruct the desired field. Farazmand

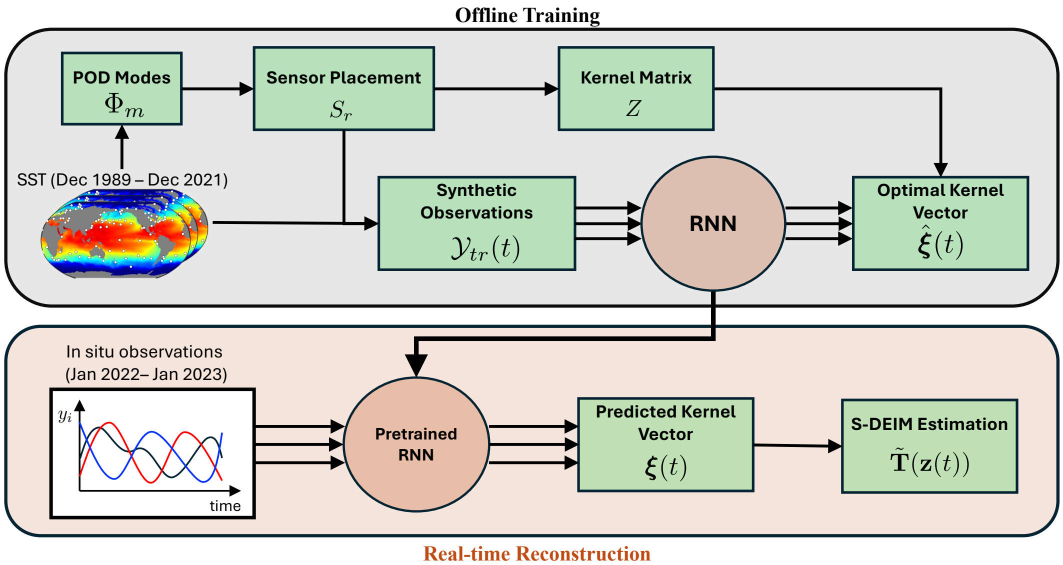

Figure 1: Schematic framework for S-DEIM. In the offline stage, we use high-fidelity data to train the RNN. This pretrained RNN is then used in real-time reconstruction to obtain the S-DEIM estimate from sparse in situ observations.

(2024) introduced S-DEIM, extending DEIM to the underdetermined case ($r < m$). S-DEIM adds the previously ignored kernel vector to the solution of the least squares problem underlying DEIM. Unfortunately, the optimal value of this kernel cannot be evaluated from sensor measurements alone. Farazmand (2024) introduced a data assimilation method that leverages the governing equations of the system to estimate this unknown kernel vector. More recently, it was also demonstrated that this kernel vector can be estimated as the output of a recurrent neural network whose inputs comprise the time series of the observational data, including past history (Farazmand, 2026). As a result, S-DEIM can be used as an model-free data assimilation method. This RNN-based S-DEIM has proven extremely effective on synthetic data, i.e., data produced as numerical solutions of differential equations (Farazmand, 2026). The present paper is the first time that its efficacy is being tested on truly empirical data, i.e., NOAA's sea surface temperature data.

1.2 Outline

The structure of the paper is as follows. In Section 2, we describe the datasets used in this paper. Section 3 reviews S-DEIM and outlines the sensor placement methods. In Section 4, we discuss estimation of the S-DEIM kernel vector using recurrent neural networks. We present the numerical results in Section 5. Finally, Section 6 concludes our findings and presents directions for future work.

2 Datasets Used

The proposed framework is depicted in Figure 1. Each compartment of this framework is elaborated in Sections 3 and 4. It consists of two primary components: the offline training phase which requires high-fidelity SST data and a real-time reconstruction phase which only relies on sparse in situ observations.

Throughout this paper, we use the weekly NOAA Optimal Interpolation SST (OISST) dataset, spanning the period from December 1989 to January 2023 (Reynolds et al., 2002). We split this dataset into two subsets:

1. Training data (December 1989 - December 2021): This comprises our high-fidelity data for the offline training phase. The weekly SST is stored on a spatial grid of 360×180, i.e., one grid point per degree in both longitudinal and latitudinal directions.
2. Test data (January 2022 - January 2023): This data is used to extract observational time series at the sensor locations. Only these sparse time series, not the high-fidelity data, are used to obtain the S-DEIM reconstructions. Therefore, the test data should be interpreted as a proxy for in situ observations, when high-fidelity SST data is unavailable. The high-fidelity test data are only used on the backend to compute and report reconstruction errors.

In oceanography, it is customary to remove the temporal average of SST and work with SST anomalies (Deser et al., 2010). Here, we follow this convention by computing the SST time-average of the training data and removing it from both training and test data. Centering the data through this procedure also helps improve the quality of the extracted POD modes (Berkooz et al., 1993). The temperature can be easily recovered by adding the training mean back to the reconstructed SST anomalies. In the following sections, we simply say SST instead of SST anomaly, unless otherwise stated explicitly.

3 Methods

Consider the sea surface temperature anomalies $T(\mathbf{x}, t)$, where t denotes time and $\mathbf{x} \in \Omega$ is a point on the surface of the globe Ω. The NOAA dataset is discretized over the spatial grid $\mathcal{G}$ which contains 360×180 grid points, one grid point per degree. We denote the grid points with $\mathcal{G} := \{\mathbf{x}_i\}_{i=1}^N \subset \Omega$ where $N = 44,219$, excluding the grid points that fall on land. At each time t, we store the SST as a vector,

$$\mathbf{T}(t) := [T(\mathbf{x}_1, t), T(\mathbf{x}_2, t), \ldots, T(\mathbf{x}_N, t)]^\top \in \mathbb{R}^N. \tag{1}$$

Our goal is to infer the global SST $\mathbf{T}(t)$ from in situ observations. More precisely, we assume only r entries of $\mathbf{T}(t)$ are known through direct observations where $r \ll N$. In practice, the known measurements are obtained from r sensors, e.g., buoys. Let $\{i_k\}_{k=1}^r \subset \{1, 2, \ldots, N\}$ denote the indices of the sensor locations. We collect the observations in the vector $\mathbf{y}(t) := [\mathbf{T}_{i_1}(t), \ldots, \mathbf{T}_{i_r}(t)]^\top + \boldsymbol{\epsilon}$ where $\boldsymbol{\epsilon} \in \mathbb{R}^r$ represents observational noise. For simplicity, we neglect observational noise in this section and set $\boldsymbol{\epsilon} = \mathbf{0}$.

We encode the location of r sensors in a *selection matrix* $S_r := [\mathbf{e}_{i_1} | \mathbf{e}_{i_2} | \ldots | \mathbf{e}_{i_r}]$, where $\{\mathbf{e}_i\}_{i=1}^N$ denote the standard Euclidean bases in $\mathbb{R}^N$. Then, the observations at time t can be expressed as

$$\mathbf{y}(t) = S_r^\top \mathbf{T}(t). \tag{2}$$

We seek to estimate the complete state $\mathbf{T}(t)$ from these observations $\mathbf{y}(t)$. For notational simplicity, we may omit the dependence of variables on time t. We estimate the vector $\mathbf{T}$ within an orthonormal set $\{\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_m\} \subset \mathbb{R}^N$ so that

$$\mathbf{T}(t) \simeq \sum_{i=1}^{m} \alpha_i(t) \boldsymbol{\phi}_i = \Phi_m \boldsymbol{\alpha}(t). \tag{3}$$

Here, $\Phi_m := [\boldsymbol{\phi}_1 | \boldsymbol{\phi}_2 | \ldots | \boldsymbol{\phi}_m] \in \mathbb{R}^{N \times m}$ is the *basis matrix* with m basis vectors, and the coefficients $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \ldots, \alpha_m]^\top \in \mathbb{R}^m$ represent coordinates in the basis $\{\boldsymbol{\phi}_i\}_{i=1}^m$.

As we discuss in Section 3.2, the basis matrix is usually derived from POD analysis of the training dataset. As the basis matrix Φ_m is truncated to include only m modes, any reconstruction $\Phi_m \boldsymbol{\alpha}$ incurs some truncation error,

$$\mathcal{E}_m(\boldsymbol{\alpha}) := \|\Phi_m \boldsymbol{\alpha} - \mathbf{T}\|, \tag{4}$$

where $\|\cdot\|$ denotes the standard Euclidean norm. Consider the coefficients $\boldsymbol{\alpha}$ that minimize the truncation error,

$$\hat{\boldsymbol{\alpha}} := \arg \min_{\boldsymbol{\alpha} \in \mathbb{R}^m} \mathcal{E}_m(\boldsymbol{\alpha}), \tag{5}$$

where the unique minimizer is given by $\hat{\boldsymbol{\alpha}} = \Phi_m^\top \mathbf{T}$. The coefficients $\hat{\boldsymbol{\alpha}}$ determine the *best fit approximation*,

$$\hat{\mathbf{T}} := \Phi_m \hat{\boldsymbol{\alpha}} = \Phi_m \Phi_m^\top \mathbf{T}, \tag{6}$$

which represents the orthogonal projection of $\mathbf{T}$ onto the subspace spanned by the columns of the basis matrix Φ_m. The truncation error $\mathcal{E}_m(\hat{\boldsymbol{\alpha}})$ decreases monotonically as the number of modes m increases.

3.1 S-DEIM

Unfortunately, the best fit approximation (6) is not directly computable because it requires knowledge of the complete SST $\mathbf{T}$. Instead, we define the observation error,

$$\mathcal{E}_{obs}(\boldsymbol{\alpha}) := \left\| S_r^\top \Phi_m \boldsymbol{\alpha} - \mathbf{y} \right\|. \tag{7}$$

Note that $S_r^\top \Phi_m \boldsymbol{\alpha}$ corresponds to the entries of the reconstruction $\Phi_m \boldsymbol{\alpha}$ at the sensor locations. Since the observations $\mathbf{y}$ are known, we can compute the observation error $\mathcal{E}_{obs}$ for a given $\boldsymbol{\alpha}$. To form the reconstruction $\tilde{\mathbf{T}} = \Phi_m \boldsymbol{\alpha}$, we identify the coefficients $\boldsymbol{\alpha}$ that minimize the observation error. The general solution is given by

$$\left(S_r^\top \Phi_m\right)^+ \mathbf{y} + \mathbf{z} = \arg\min_{\boldsymbol{\alpha} \in \mathbb{R}^m} \mathcal{E}_{obs}(\boldsymbol{\alpha}), \quad \forall \mathbf{z} \in \mathcal{N}\left[S_r^\top \Phi_m\right]. \tag{8}$$

The superscript $+$ denotes the Moore–Penrose pseudo-inverse and $\mathcal{N}$ represents the null space, so that $\mathcal{N}[S_r^\top \Phi_m] = \left\{\mathbf{z} \in \mathbb{R}^m : S_r^\top \Phi_m \mathbf{z} = \mathbf{0}\right\}$. We refer to $\mathbf{z}$ as the *kernel vector*, an arbitrary vector within the null space of $S_r^\top \Phi_m$. The minimizer (8) yields the *S-DEIM reconstruction* (Farazmand, 2024),

$$\tilde{\mathbf{T}}(\mathbf{z}) := \Phi_m \left(S_r^\top \Phi_m\right)^+ \mathbf{y} + \Phi_m \mathbf{z}, \quad \mathbf{z} \in \mathcal{N}\left[S_r^\top \Phi_m\right]. \tag{9}$$

DEIM and Q-DEIM use the solution with the trivial kernel vector, $\mathbf{z} = \mathbf{0}$, leading to the reconstruction $\tilde{\mathbf{T}} = \Phi_m \left(S_r^\top \Phi_m\right)^+ \mathbf{y}$ (Chaturantabut & Sorensen, 2010; Drmač & Gugercin, 2016). In fact, when the number of sensors is greater than or equal to the number of modes ($r \geq m$), the null space is typically trivial, $\mathcal{N}\left[S_r^\top \Phi_m\right] = \{\mathbf{0}\}$, and the (Q-)DEIM reconstruction is the only choice. However, in the underdetermined regime, where the number of sensors is smaller than the number of modes ($r < m$), nonzero kernel vectors become an option. This regime occurs in practice when sensors are limited in numbers by cost and deployment obstacles, yet a larger model complexity ($m > r$) is needed to satisfactorily approximate the SST within the basis Φ_m. Here, we are mainly interested in this underdetermined case.

In this regime, one needs to answer the following question: is there an optimal kernel vector $\mathbf{z} \in \mathcal{N}[S_r^\top \Phi_m]$ that returns the best approximation $\tilde{\mathbf{T}}(\mathbf{z})$ of $\mathbf{T}$? It is straightforward to verify that the observation error (7) is independent of the kernel vector. In other words, the instantaneous observations $\mathbf{y}(t)$ contain no additional information that would inform the choice of an optimal kernel vector (Farazmand, 2024). However, we may ask whether there exists a kernel vector $\mathbf{z}$ which minimizes the *total error*,

$$\mathcal{E}_{tot}(\mathbf{z}(t)) := \left\| \tilde{\mathbf{T}}(\mathbf{z}(t)) - \mathbf{T}(t) \right\|. \tag{10}$$

To answer this question, consider an orthonormal basis $\{\mathbf{z}_1, \ldots, \mathbf{z}_{m-r}\}$ of the null space $\mathcal{N}\left[S_r^\top \Phi_m\right]$, and define the *kernel matrix* $Z := [\mathbf{z}_1|\ldots|\mathbf{z}_{m-r}] \in \mathbb{R}^{m\times(m-r)}$. Then, there exists a unique *optimal kernel vector*,

$$\hat{\mathbf{z}}(t) := ZZ^\top \Phi_m^\top \mathbf{T}(t) = \underset{\mathbf{z}\in\mathcal{N}[S_r^\top \Phi_m]}{\arg\min}\ \mathcal{E}_{tot}(\mathbf{z}), \tag{11}$$

which minimizes the total error (Farazmand, 2024). Although the kernel matrix Z can be easily computed, the optimal kernel vector $\hat{\mathbf{z}}$ requires knowledge of the full SST $\mathbf{T}(t)$, and therefore is not directly computable. It appears that we might have hit an impasse again. However, as we discuss in Section 4, a machine learning method can be used to estimate the optimal kernel vector from sparse observational time series. This machine learning method leverages the past history of the observations $\mathbf{y}(s)$, for $s \leq t$, in order to estimate the optimal kernel vector $\hat{\mathbf{z}}(t)$ at the current time t. But before discussing the machine learning method, we first review the processes for determining the basis matrix Φ_m (Section 3.2) and the selection matrix S_r (Section 3.3).

3.2 Basis Matrix

The basis matrix Φ_m is obtained using Proper Orthogonal Decomposition (POD) (Lumey, 1975; Sirovich, 1987), also known as Principal Component Analysis (PCA) (Hotelling, 1933; Pearson, 1901). First, we form the data matrix,

$$A := [\mathbf{T}(t_1)|\mathbf{T}(t_2)|\ldots|\mathbf{T}(t_M)] \in \mathbb{R}^{N\times M}, \tag{12}$$

from the SST anomalies in the training dataset. As detailed in Section 2, each time instance t_i corresponds to a week from Dec. 1989 to Dec. 2021, resulting in a total of $M = 1670$ snapshots. We compute the Singular Value Decomposition (SVD) of A, where the $N\times N$ matrix $\Phi = [\boldsymbol{\phi}_1|\boldsymbol{\phi}_2|\ldots|\boldsymbol{\phi}_N]$ denotes the left singular matrix and $\{\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_N\}$ are the left singular vectors or *POD modes*. We truncate the left singular matrix to the first m modes in order to obtain our basis matrix $\Phi_m = [\boldsymbol{\phi}_1|\boldsymbol{\phi}_2|\ldots|\boldsymbol{\phi}_m]$. The m-dimensional linear subspace which best approximates the training dataset is spanned by the columns of Φ_m (Koch & Lubich, 2007).

Figure 2 shows three POD modes, emphasizing that they often encode interpretable information about the SST. The first POD mode delineates the southern and northern hemispheres, whereas coastline features are captured in the third mode, and the Pacific South Equatorial Current is clearly represented in the fourth mode. We emphasize that, being normalized singular vectors, the numerical values of POD modes do not represent true temperature values and should not be interpreted to indicate hot/cold temperatures; they only encode the underlying patterns within the data.

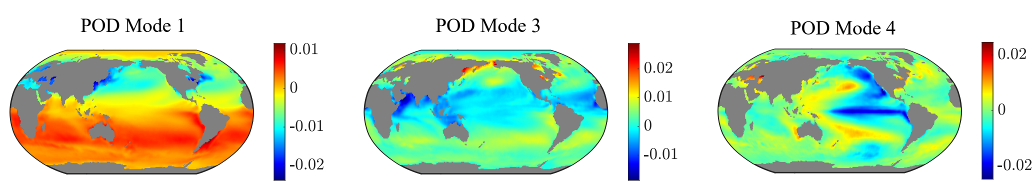

Figure 2: Three POD modes generated using the training dataset.

3.3 Sensor placement

In our numerical results (Section 5), we consider two types of sensor placement: random and CPQR. The former mimics the practical situations where one does not have control over the locations of the in situ observations. The latter is performed for comparison and to demonstrate the robustness of S-DEIM regardless of the sensor placement method.

When sensor placement is random, we form the selection matrix $S_r = [\mathbf{e}_{i_1}|\mathbf{e}_{i_2}|\dots|\mathbf{e}_{i_r}]$ by drawing r distinct indices $\{i_1, i_2, \dots, i_r\}$ randomly from the set of possible indices $\{1, 2, \dots, N\}$.

We also briefly review the CPQR method for sensor placement. This method was first proposed by Drmač and Gugercin (2016) under the assumption that the number of modes and sensors are equal ($r = m$) and was later extended to the case when $r < m$ (Kakasenko et al., 2025). We refer to Drmač and Gugercin (2016) and Kakasenko et al. (2025) for a detailed discussion of the CPQR algorithm.

Consider the column-pivoted QR decomposition of $\Phi_r^\top$,

$$\Phi_r^\top \Pi = QR, \quad \Pi = \begin{bmatrix} \Pi_1 & \Pi_2 \end{bmatrix}, \quad R = \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix}, \tag{13}$$

where Π is a permutation matrix with $\Pi_1 \in \mathbb{R}^{N\times r}$ and $\Pi_2 \in \mathbb{R}^{N\times(N-r)}$, $Q \in \mathbb{R}^{r\times r}$ is an orthogonal matrix, and $R_{11} \in \mathbb{R}^{r\times r}$ is an upper triangular matrix. In CPQR sensor placement, we set the selection matrix $S_r = \Pi_1$, the first r columns of the permutation matrix Π. In equation (13), we used the decomposition of the matrix Φ_r, the first r columns of the full basis matrix Φ_m. This choice is purely based on the empirical observation that it returned more accurate reconstructions compared to CPQR sensor placement based on Φ_m.

If the selection matrix S_r is obtained from the CPQR decomposition, the resulting matrix $S_r^\top \Phi_m$ has full rank (Gu & Eisenstat, 1996), so that $\dim(\mathcal{N}[S_r^\top \Phi_m]) = m - r$. However, when the sensor locations are randomly selected, $S_r^\top \Phi_m$ is not guaranteed to have full rank. Nonetheless, we observed that $S_r^\top \Phi_m$ tends to be full-rank in practice, even when the sensors are placed randomly.

Finally, we highlight an implicit assumption in our sensor placement methods, namely that the sensor locations are static. This allows in situ observations from moored buoys but excludes observations from drifters and ships that move over time. Moving sensors would imply a time-dependent selection matrix $S_r(t)$, which in turn implies a time-dependent kernel matrix $Z(t)$. The current S-DEIM theory (Farazmand, 2024, 2026) is not suitable for such time-dependent cases whose handling requires further theoretical developments.

4 S-DEIM with Recurrent Neural Networks

We recall that the S-DEIM reconstructions $\tilde{\mathbf{T}}(\mathbf{z})$ are most accurate when the optimal kernel vector $\mathbf{z} = \hat{\mathbf{z}}$ is used; see Eq. (11). But computing the optimal kernel vector $\hat{\mathbf{z}}$ requires knowledge of the entire temperature field $\mathbf{T}$ which is unavailable. Therefore, the feasibility of S-DEIM for SST reconstruction hinges on our ability to estimate the optimal kernel vector from sparse in situ data $\mathbf{y}$. To this end, we utilize recurrent neural networks (RNNs) to learn the mapping from the in situ observational time series to the optimal kernel vector. An RNN is a type of machine learning model particularly suited for sequential time series data. Its internal memory allows it to learn the time-dependent patterns linking the time series of sparse sensor readings to the optimal kernel coordinates. The RNN takes a time series of past observations,

$$\mathcal{Y}(t) = \{\mathbf{y}(s) : s \leq t\}, \tag{14}$$

as inputs to predict the optimal kernel vector $\hat{\mathbf{z}}(t)$ as its output. As discussed in Farazmand (2026), it is necessary to use the past history of in situ observations, since instantaneous observations $\mathbf{y}(t)$ alone do not contain enough information to infer the optimal kernel vector $\hat{\mathbf{z}}(t)$.

RC		LSTM	
Parameter	**Value**	**Parameter**	**Value**
Reservoir Size (N_r)	100	Hidden Units (h_t)	300
Learning Rate (α)	1.0	Initial Learning Rate (α)	0.01
Network Sparsity (s)	40%	Drop Period (τ)	50
Regularization Parameter (λ)	10^{-8}	Drop Rate (γ)	0.1

Table 1: Hyperparameters for the two types of RNN architectures used in this paper: RC and LSTM.

As depicted in Figure 1, our framework consists of an offline training stage and a real-time reconstruction stage. The offline stage uses historical high-fidelity SST data to prepare the necessary components for S-DEIM reconstruction and for training the RNN. In particular, in the offline stage, we compute the POD basis matrix (Φ_m) and the selection matrix (S_r) for sensor placement. Furthermore, we use the high-fidelity SST data to prepare the training pairs, which consists of input time series $\mathcal{Y}_{tr}(t)$ and the corresponding target output $\hat{\mathbf{z}}(t)$. We use these input-output pairs to train the RNN. Once trained, the RNN can be deployed for real-time reconstruction. In this stage, the pretrained RNN takes the available sparse in situ observations $\mathcal{Y}(t)$ to predict the kernel vector $\mathbf{z}(t)$. This predicted vector is then used for the S-DEIM reconstruction (9) of the SST.

An important practical note is in order here. The RNN can, in principle, be trained to predict the m-dimensional optimal kernel vector $\hat{\mathbf{z}}(t) \in \mathcal{N}[S_r^\top \Phi_m] \subset \mathbb{R}^m$. However, without loss of generality, the dimension of the RNN output can be decreased, thus improving the performance of the RNN (Farazmand, 2026). To this end, recall that the columns of the kernel matrix Z form an orthonormal basis for the null space $\mathcal{N}[S_r^\top \Phi_m]$. Therefore, we can write $\hat{\mathbf{z}}(t) = Z\hat{\boldsymbol{\xi}}(t)$, where the unique vector $\hat{\boldsymbol{\xi}}(t) \in \mathbb{R}^{m-r}$,

$$\hat{\boldsymbol{\xi}}(t) = Z^\top \hat{\mathbf{z}}(t) = Z^\top \Phi_m^\top \mathbf{T}(t), \tag{15}$$

denotes the coordinates of $\hat{\mathbf{z}}(t) \in \mathbb{R}^m$ in the basis Z. Since the dimension of the coordinate vector $\hat{\boldsymbol{\xi}}(t) \in \mathbb{R}^{m-r}$ is smaller than that of the full kernel vector $\hat{\mathbf{z}}(t) \in \mathbb{R}^m$, it is more efficient to train the network to predict $\hat{\boldsymbol{\xi}}(t)$. Subsequently, the kernel vector $\hat{\mathbf{z}}(t) = Z\hat{\boldsymbol{\xi}}(t)$ can be recovered uniquely from the kernel coefficients $\hat{\boldsymbol{\xi}}(t)$.

In Section 5, we report the SST reconstruction results for two types of RNNs: the Reservoir Computing (RC) network and the Long Short-Term Memory (LSTM) network. The architecture of each network and the hyperparameter tuning process are detailed in Appendix A. The eventual hyperparameters that we used for the RNNs are detailed in Table 1.

Table 2 reports the computational time it takes to train each RNN and the inference time on the test dataset, i.e., computational time it takes for the pretrained RNN to produce SST reconstructions from in situ observations. The computations are carried out on a workstation with a Windows 11 operating system, Intel Core i9-14900K CPUs, and 32 GB memory.

For the LSTM network, the training process takes approximately one minute, whereas for the RC network the training time is significantly less, around 0.01 seconds. The inference time for both network architectures is less than a second.

	Training	Inference
RC	0.01	0.03
LSTM	61.15	0.32

Table 2: Wall clock time for training and inference for each RNN architecture. The units of time are in seconds.

5 Results and Discussion

In this section, we demonstrate the effectiveness of S-DEIM in reconstructing SST snapshots from partial observations. We form S-DEIM reconstructions using two types of RNNs (RC and LSTM) to compute the kernel vector. We compare these results against Q-DEIM reconstructions, which use the trivial kernel vector $\mathbf{z} = \mathbf{0}$. In addition, we report the S-DEIM reconstruction using the optimal kernel vector $\hat{\mathbf{z}}$ as well as the best fit approximation $\hat{\mathbf{T}}$. Recall that the S-DEIM reconstruction with the optimal kernel vector and the best fit approximation are not computable in practice, since they require access to the full SST. Nevertheless, they are reported here since they provide a comparison baseline for the best possible approximations.

We use the weekly NOAA Optimal Interpolation SST dataset (OISST) spanning the period from December 1989 to January 2023 (Reynolds et al., 2002). As outlined in Section 2, we divide the dataset into two subsets: training (December 1989 - December 2021) and testing (January 2022 - January 2023). Each SST snapshot is given on a 360×180 grid, recording the SST in increments of one degree. We vectorize each snapshot, discarding the grid points that fall on land. In addition, we center the data by removing the temporal mean of the training data from all SST snapshots. The reported relative errors refer to the relative error of a reconstruction $\tilde{\mathbf{T}}$ compared to the truth $\mathbf{T}$,

$$RE = \frac{\|\tilde{\mathbf{T}} - \mathbf{T}\|}{\|\mathbf{T}\|}, \tag{16}$$

where all temperature vectors refer to mean-removed SST (i.e., SST anomalies).

Throughout this section, the reconstructions $\tilde{\mathbf{T}}$ are obtained from $r = 100$ in situ observations, which represents a very sparse distribution; roughly 0.2% of the high-resolution grid. The number of sensors $r = 100$, although somewhat arbitrary, is close to the number of NOAA's moored buoys available in the National Data Buoy Center (NDBC) (McCall, 1998).

As the quality of the reconstruction is dependent on the sensor locations, we also compare the reconstructions for different methods of sensor selection. Section 5.1 discusses the SST reconstructions from CPQR sensor placement. In Section 5.2, we also report results from random sensor placement, demonstrating the robustness of S-DEIM reconstructions. In particular, we demonstrate that S-DEIM is not as sensitive to sensor placement method as DEIM. Note that the SST data used for our computations is derived from observational data which inherently include noise. Even so, adding an additional 5% synthetic Gaussian noise to the observations did not change the results appreciably.

5.1 Reconstruction from CPQR Sensor Placement

In this section, we present the S-DEIM reconstructions resulting from CPQR sensor placement (see Section 3.3). We first examine the reconstruction error for a fixed num-

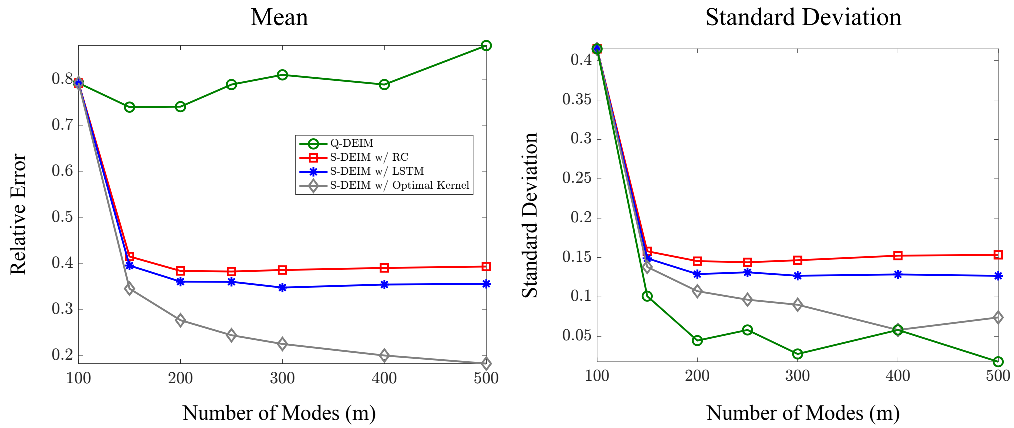

Figure 3: Mean and standard deviation of relative reconstruction error for an increasing number of modes m. The mean and standard deviation are taken over the snapshots in the test dataset.

ber of sensors, $r = 100$, as the number of modes m increases. Then we analyze the temporal and spatial distribution of the errors.

Varying number of modes: Figure 3 shows the mean relative error and standard deviation of the reconstructions, computed from the test data (Jan. 2022–Jan. 2023). The relative error of the Q-DEIM reconstructions, which use the trivial kernel vector $\mathbf{z} = \mathbf{0}$, fluctuates near 80%. As the number of modes increases, the error decreases slightly at first, but then increases for $m > 200$, so that no particular trend is observed for the Q-DEIM reconstruction error.

Next we turn to the S-DEIM reconstructions using RC and LSTM neural networks. Note that when $m = 100$, the number of modes matches the number of sensors, resulting in a trivial null space $\mathcal{N}[S_r^\top \Phi_m] = \{\mathbf{0}\}$. Therefore, the Q-DEIM and S-DEIM reconstructions are equivalent when $m = r = 100$. However, when $m > r$ the S-DEIM reconstructions significantly outperform Q-DEIM. More specifically, we observe an immediate drop in the error of the S-DEIM reconstructions, from 79.30% with $m = 100$ modes to approximately 40% with $m = 150$ modes. The error then continues to decline to approximately 36% as the number of modes increases from 150 to 300. At this point, the S-DEIM errors begin to plateau; further increasing the number of modes, beyond $m = 300$, does not result in significantly higher accuracy.

For reference, we also show the relative error of S-DEIM reconstructions with the optimal kernel vector $\hat{\mathbf{z}}$, but recall that these reconstructions are not computable from sparse observations. It is noteworthy that there is an appreciable gap between the S-DEIM reconstructions using RNNs and the optimal S-DEIM reconstructions. In theory, this gap can be closed with more training data and more complex network architectures. But in practice, this gap is expected due to limited training data, training error (inability to find the global loss minimizer using numerical optimization), and the limited expressivity of a finite network (Bottou & Bousquet, 2007; Welper & Keene, 2025).

Turning our attention to the standard deviation of the relative error, we observe a similar trend. For S-DEIM reconstructions, there is a sharp drop in the standard deviation of the error from $\sigma = 0.41$ for $m = 100$ modes to about $\sigma = 0.13$ for $m = 150$ modes. The standard deviation also plateaus eventually as m increases. Interestingly,

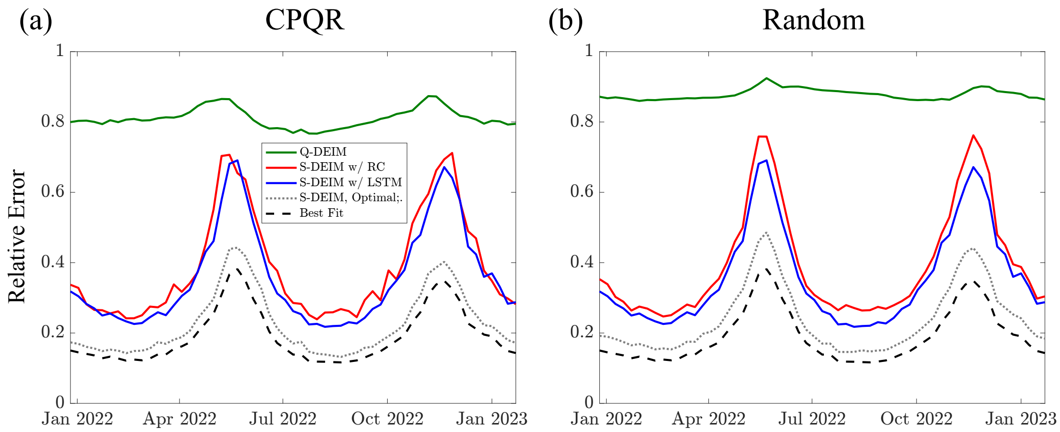

Figure 4: Relative reconstruction error with $r = 100$ sensors and $m = 300$ modes. (a) CPQR sensor placement. (b) Randomly placed sensors.

Q-DEIM's standard deviations are the lowest. However, given the large mean relative error of Q-DEIM, these relatively small deviations should not be confused with better performance.

Due to diminishing returns in performance and the increasing computational cost of going beyond 300 modes, we fix the number of modes at $m = 300$ to further analyze the S-DEIM reconstructions. Although Q-DEIM is usually performed with an equal number of sensors and modes, here the Q-DEIM error for $m = 100$ is similar to the error for $m = 300$ (see Figure 3). Therefore, we use Q-DEIM with $m = 300$ modes and $r = 100$ sensors for comparison, which puts it on the same footing as the S-DEIM reconstructions.

Temporal evolution: Figure 4(a) shows the weekly reconstruction error for the test data (Jan. 2022-Jan. 2023). The S-DEIM reconstructions are significantly better than Q-DEIM for every time instance. Specifically, the LSTM- and RC-based S-DEIM reconstructions achieve mean relative errors of 34.82% and 38.64%, respectively, which are more than 40% lower than Q-DEIM's error of 81.08%. The best fit and optimal S-DEIM reconstruction errors are also shown in Figure 4 for reference, but recall that they are not computable from sparse observational data. Maximum reconstruction errors occur during seasonal transitions in May and November, where S-DEIM reconstruction error peaks at approximately 68%. This suggests that the seasonal shifts may require a higher number of POD modes to accurately capture.

Spatial distribution: Next, we consider the variability in reconstruction errors between S-DEIM and Q-DEIM. We compute the difference ΔT between the reconstruction $\tilde{T}$ and the true SST T at each grid point $\mathbf{x}_i$ and each time instance t_j, where $\Delta T(\mathbf{x}_i, t_j) = \tilde{T}(\mathbf{x}_i, t_j) - T(\mathbf{x}_i, t_j)$. In Figure 5, we compare the Probability Density Function (PDF) of ΔT for Q-DEIM and LSTM-based S-DEIM. The PDF for the RC-based S-DEIM is nearly identical to that of the LSTM-based S-DEIM and is therefore omitted. The S-DEIM error ΔT exhibits significantly smaller variance, with a narrower range from approximately $-5°$C to $4°$C. In contrast, the Q-DEIM errors span a wider range, from around $-15°$C to $10°$C. Furthermore, the S-DEIM errors are highly concentrated near zero, with 91% falling within $\pm 1°$C of the truth. In contrast, only 51% of Q-DEIM reconstructions are within this range. Hence, S-DEIM is much less prone to extreme errors in both frequency and intensity compared to Q-DEIM.

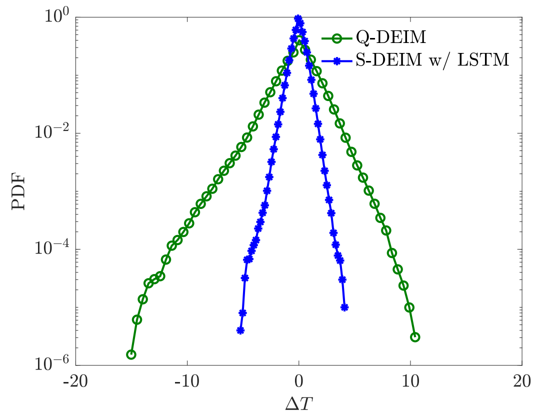

Figure 5: The PDF of the difference ΔT between the reconstructed SST and the truth for Q-DEIM and S-DEIM. For S-DEIM, ΔT is concentrated around 0°C, with 91% of reconstructions falling within ±1°C. For Q-DEIM, ΔT exhibits heavier tails and only 51% are within ±1°C.

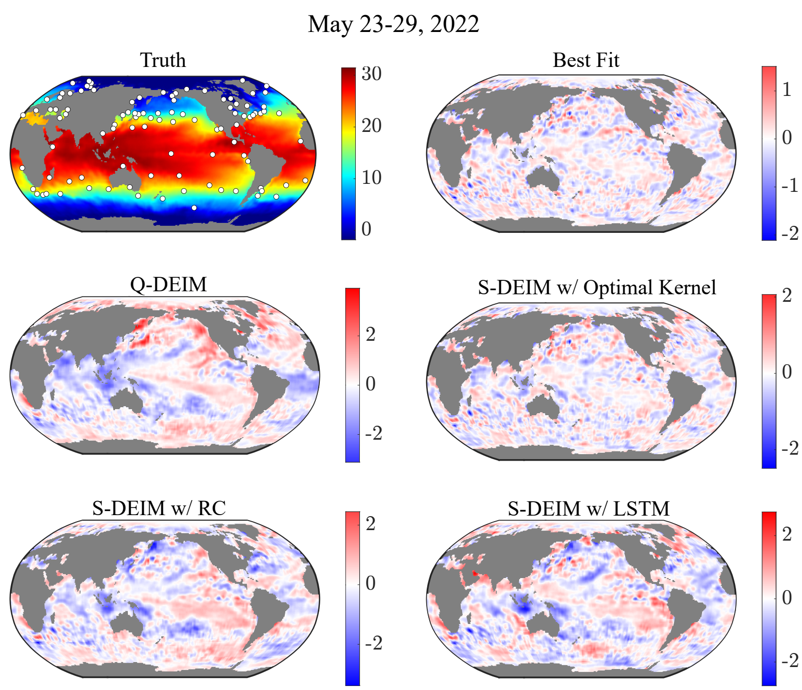

Figure 6: True SST for the week of May 23-29, 2022 together with sensor locations marked by white circles (top left). Other panels show the difference ΔT between each reconstruction and the true SST. May 23-29, 2022 is the week in which the S-DEIM reconstructions have maximum error.

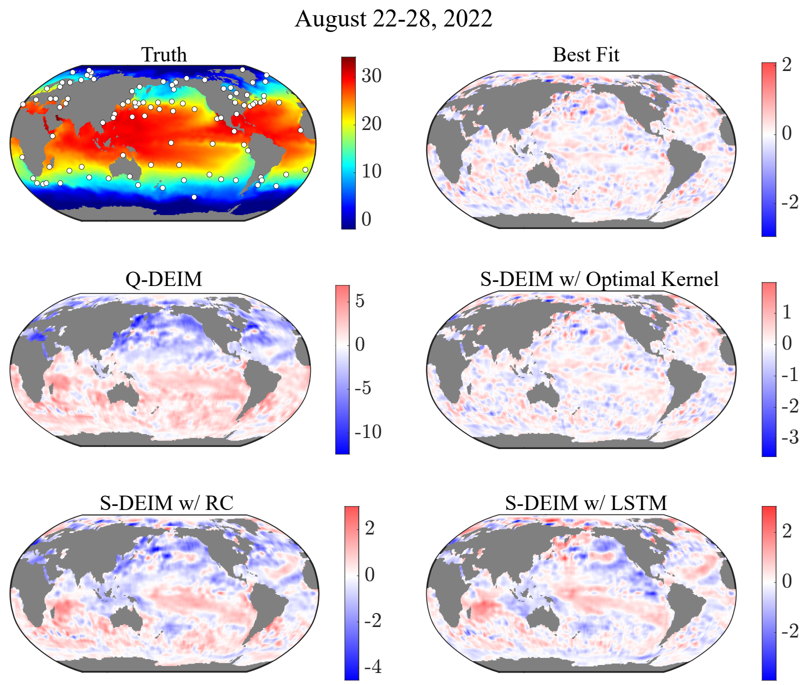

Figure 7: True SST for the week of August 22-28, 2022 together with sensor locations marked by white circles (top left). Other panels show the difference ΔT between each reconstruction and the true SST. August 22-28, 2022 is the week in which the S-DEIM reconstructions have minimum error.

We select two representative weeks to showcase the spatial distribution of error in our reconstructions. Figures 6 and 7 compare the reconstructions to the true SST during the weeks of maximum and minimum error, respectively. More precisely, Figure 6 corresponds to the week of May 23-29, 2022 and Figure 7 corresponds to August 22-28, 2022. The upper left panel in each figure displays the true SST field (not anomalies) as well as the CPQR sensor locations marked with white circles. The remaining panels show the difference ΔT between the reconstructed field $\tilde{T}$ and the true temperature field T, i.e., $\Delta T = \tilde{T} - T$.

The top right panel is the best fit approximation and the center right panel shows the optimal S-DEIM reconstruction. These two are only included for reference, since they are not computable from sparse observations. The optimal S-DEIM and best fit approximations show small and spatially dispersed errors, whereas the RC (bottom left) and LSTM (bottom right) S-DEIM reconstructions exhibit a more structured error distribution. For example, in Figure 6, both RNN-based S-DEIM reconstructions underestimate the SST in the Indonesian region by approximately 2°C. Figure 7 shows a similar underestimation across the Western Pacific Ocean. More generally, across all snapshots, we found a few locations where the maximum S-DEIM reconstruction errors tend to cluster: along the Kuroshio current, along the Agulhas Current, and along the Pacific Ocean's South Equatorial Current. In contrast, the Q-DEIM reconstruction errors are more uniformly distributed around the globe (see, e.g., center-left panel of Figure 7).

In summary, with CPQR sensor placement, S-DEIM returns significantly better reconstructions compared to Q-DEIM. The computational cost of S-DEIM is only slightly higher than Q-DEIM. The primary computational cost lies in the one-time offline train-

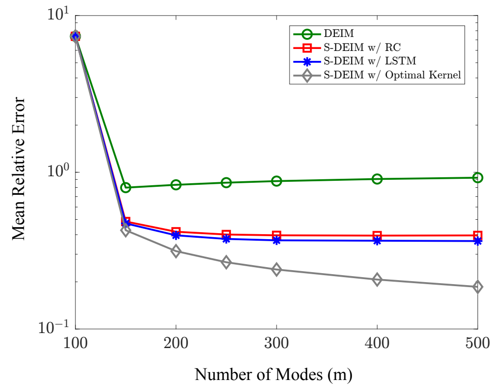

Figure 8: Mean reconstruction error averaged over 25 different realizations of randomly placed sensors as a function of the number of modes.

ing of the RNN. Once trained, the S-DEIM inference of the SST takes less than a second; see Table 2.

5.2 Reconstruction from Random Sensor Placement

In practice, the location of in situ observations is determined based on logistical considerations as opposed to the optimality of the resulting SST reconstructions. To emulate this practical situation, we consider SST reconstructions from randomly placed sensors. More precisely, we consider 25 realizations of $r = 100$ sensors, placed at random points on the sea surface. For each realization, we compute DEIM and S-DEIM reconstructions and assess the sensitivity of different reconstruction methods to the sensor placement method.

As discussed in Section 3.3, when the sensors are placed randomly, the matrix $S_r^\top \Phi_m$ may be rank deficient. In such cases, the dimension of the null space $\mathcal{N}[S_r^\top \Phi_m]$ is greater than $m-r$. To ensure that the dimension of the null space stays consistent, we discard selection matrices S_r for which $S_r^\top \Phi_m$ does not have full rank. This guarantees that $\mathrm{rank}(S_r^\top \Phi_m) = r$ across all 25 realizations of random sensors.

Figure 8 shows the mean relative error, where the average is taken over all test data (Jan. 2022–Jan. 2023) and all 25 realizations of randomly placed sensors. We plot the mean relative error as a function of the number of modes m. This figure is analogous to Fig. 3 where the sensors were placed based on the CPQR algorithm. We first discuss the DEIM reconstructions. For the same number of sensors and modes ($m = r = 100$), DEIM performs very poorly with an average relative error of approximately 737%. Compare this to the Q-DEIM error of 79.30% in Fig. 3 where CPQR sensor placement was used. This observation is in line with previous studies that report similarly large errors for DEIM when sensors are randomly placed (Manohar et al., 2018).

Interestingly, as the number of modes increases from $m = 100$ to $m = 150$, there is a sharp drop in average relative error for all reconstruction methods. In particular, the DEIM error drops an order of magnitude to 79.83%, which is similar to the Q-DEIM reconstruction errors with CPQR sensor placement. In summary, DEIM reconstructions from randomly placed sensors are rather abysmal when the same number of sensors and modes is used. However, when the number of modes is increased while keeping the number of sensors fixed, the DEIM reconstructions with random sensor placement are almost as accurate as the Q-DEIM reconstructions with CPQR sensor placement.

Now we turn our attention to the S-DEIM reconstructions. Recall that for $m = r$, S-DEIM and DEIM reconstructions coincide; both use the trivial kernel vector $\mathbf{z} = \mathbf{0}$. However, when $m > r$, the two differ in that S-DEIM uses an estimation of the optimal kernel vector $\hat{\mathbf{z}}$. In Figure 8, we observe that the S-DEIM reconstruction error decays rapidly as soon as the number of modes is larger than the number of sensors. We also note that S-DEIM reconstructions are significantly more accurate than DEIM reconstructions for any $m > r = 100$; S-DEIM errors are roughly 40% lower than DEIM errors. The S-DEIM error eventually plateaus when $m \geq 300$, where further increasing the number of modes does not lead to significant gains in terms of accuracy. Interestingly, the S-DEIM reconstruction errors are comparable to the case where CPQR sensor placement was used (cf. Figure 3). This indicates that S-DEIM reconstructions are not sensitive to the sensor placement method.

Method	Mean RE	Maximum RE
Q-DEIM (DEIM)	0.8108 (0.8787)	0.8736 (0.9246)
S-DEIM w/ RC	0.3864 (0.3962)	0.7128 (0.7621)
S-DEIM w/ LSTM	0.3482 (0.3680)	0.6367 (0.6927)
S-DEIM w/ Opt. Kernel	0.2256 (0.2397)	0.4427 (0.4862)

Table 3: Mean relative error and maximum relative error for various reconstruction methods with $m = 300$ modes and $r = 100$ sensors. The errors corresponding to CPQR sensor placement are reported next to the errors corresponding to random sensor placement (in parenthesis).

To see this more clearly, in Table 3, we report the mean and maximum relative error of the reconstructions for $m = 300$ modes. The mean is obtained by averaging the error over the entire test dataset. Similarly, the maximum is taken over the entire duration of the test dataset (Jan. 2022-Jan. 2023). DEIM with random sensor placement performs about 7% worse on average than Q-DEIM, which uses CPQR sensor placement. For randomly placed sensors, the mean relative error for S-DEIM with optimal kernel vector, RC, and LSTM are $23.97\%, 39.62\%$, and 36.80%, respectively. These errors are only about 1 to 2% higher than the mean relative error of S-DEIM with CPQR sensor placement.

Figure 4(b) shows the temporal evolution of the relative error over the period of Jan. 2022–Jan. 2023 (test dataset). The general trends are very similar to the reconstructions with CPQR sensor placement. In particular, maximum errors occur during the months of May and November where seasonal changes make predictions more challenging. These maximum errors are also reported in Table 3. Changing the sensor placement method, from CPQR to random, leads to an increase in the maximum error. However, these increases are modest at about 4 to 5% for both DEIM and S-DEIM.

6 Conclusions

Data-driven estimation of geophysical variables, such as sea surface temperature, remains a challenging task, especially when only a sparse set of observations is available. Here, we showed that Sparse Discrete Empirical Interpolation Method (S-DEIM), with the aid of recurrent neural networks, can significantly improve SST reconstructions from sparse observations. More specifically, the relative error of S-DEIM is 40% lower than its predecessors: the DEIM and Q-DEIM methods. Furthermore, 91% of S-DEIM estimates fall within $\pm 1^\circ$C of the true SST. This number is only 51% for Q-DEIM.

We demonstrated that the S-DEIM reconstructions are not unduly sensitive to sensor placement, i.e., the locations from which in situ observations are gathered. For example, S-DEIM reconstruction errors increased by only 1-2% when switching from a nearly optimal sensor placement method (i.e., CPQR) to randomly placed sensors. S-DEIM is also robust to moderate amounts of observational noise. For example, adding up to 5% Gaussian noise to the in situ observations did not change the reconstructions appreciably.

The success of S-DEIM hinges on the use of a kernel vector which is often ignored in empirical interpolation. The optimal value of this kernel vector is not computable from instantaneous in situ observations. Here, we used RNNs to estimate the optimal kernel vector from time series of past observations. Although this returned satisfactory results, there exists an approximately 10% gap between the accuracy of optimal S-DEIM and RNN-based S-DEIM reconstructions. Closing this gap could be the subject of future work. Increasing the complexity (number of layers and units) of RNNs did not close this gap, indicating that the 10% gap is likely a consequence of insufficient training data. Supplementing NOAA's SST data with synthetic data from high-fidelity simulations may close this gap.

Another direction for future work consists of reconstruction from multi-fidelity sensors (Clark et al., 2021). S-DEIM assumes that each observation is of equal quality. However, measurements collected from different sources have varying levels of uncertainty. One may address this problem, for example, by introducing a weighted norm in the least squares problem (8) underlying S-DEIM, such that lower quality observations contribute less to the residual (7).

Finally, in S-DEIM it is assumed that the sensors are static, i.e., they do not move over time. This limits us to using in situ observations from moored buoys. To enable the use of observations from drifters and ships, further theoretical developments are needed to generalize S-DEIM for use with moving sensors.

Conflict of Interest disclosure

The authors have no conflicts to disclose.

Data availability statement

The sea surface temperature data is publicly available from National Oceanic and Atmospheric Administration (NOAA) website: https://www.ncei.noaa.gov/products/optimum-interpolation-sst

Acknowledgments

This work was partially supported by the National Science Foundation (NSF) through the grant DMS-2220548 (Algorithms for Threat Detection Program). C.A., K.H., M.M., C.N., and M.F. were supported by NSF's REU program, through the award DMS-2349611. L.B.E. was supported by NSF's RTG program, through the award DMS-2342344.

Appendix A Recurrent Neural Network Architectures

We provide a high-level overview of the two Recurrent Neural Network (RNN) architectures used in this study, namely the Reservoir Computing (RC) network and the

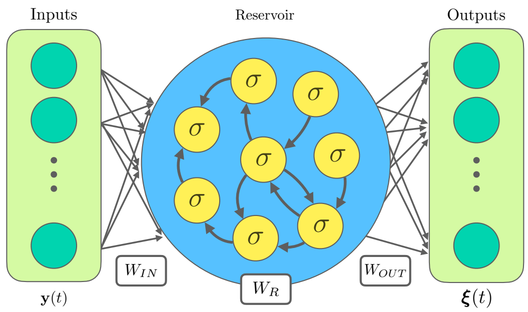

Figure A1: The RC network consists of an input layer with weights W_{IN}, a fixed random reservoir with internal weights W_R, and a trainable output layer with weights W_{OUT}.

Long Short-Term Memory (LSTM) network. We also discuss the hyperparameter tuning that was undertaken in this study.

A1 Reservoir Computing (RC)

Reservoir Computing is a specialized type of RNN notable for its training efficiency. The RC architecture consists of an input layer, a reservoir and a trainable output layer, as depicted in Figure A1. At any time t_i, we denote the reservoir state by $\mathbf{r}(t_i) \in \mathbb{R}^{N_r}$ where $N_r \gg \max\{r, m\}$ is the reservoir size, r denotes the number of sensors and m denotes the number of modes. The input data, the time series of observations $\mathbf{y}(t)$, is lifted to the high-dimensional state space of the reservoir via the input weight matrix $W_{IN} \in \mathbb{R}^{N_r \times r}$. The reservoir dynamics is regulated by its internal weights $W_R \in \mathbb{R}^{N_r \times N_r}$ and biases $\mathbf{b} \in \mathbb{R}^{N_r}$.

At each time step t_i, the reservoir state is updated with the equation,

$$\mathbf{r}(t_{i+1}) = (1-\alpha)\mathbf{r}(t_i) + \alpha\sigma(W_R\mathbf{r}(t_i) + W_{IN}\mathbf{y}(t_i) + \mathbf{b}), \tag{A1}$$

where α is the learning rate, and $\sigma(x) = \tanh(x)$ is the activation function applied element-wise.

A defining characteristic of the RC methodology is that the weight matrices for the input layer (W_{IN}), the reservoir (W_R), and the bias vector ($\mathbf{b}$) are all initialized randomly and remain fixed. We refer to Lukosevicius and Jaeger (2009) and Farazmand (2026) for details concerning the initialization of these random quantities. The only component of the network that is trained is the output weight matrix $W_{OUT} \in \mathbb{R}^{(m-r)\times N_r}$. This layer performs a linear mapping from the high-dimensional reservoir state $\mathbf{r}(t)$ to the target output, here the optimal kernel coordinates $\hat{\boldsymbol{\xi}}(t)$.

We collect the reservoir states $\mathbf{r}(t_i)$ into the matrix R and the optimal kernel coordinates $\hat{\boldsymbol{\xi}}(t_i)$ Ξ into another matrix Ξ,

$$R = [\mathbf{r}(t_1)|\mathbf{r}(t_2)|\cdots|\mathbf{r}(t_M)], \qquad \Xi = [\hat{\boldsymbol{\xi}}(t_1)|\hat{\boldsymbol{\xi}}(t_2)|\cdots|\hat{\boldsymbol{\xi}}(t_M)]. \tag{A2}$$

The optimal output weights W_{OUT} are determined by solving the regularized least-squares problem,

$$W_{OUT} = \arg\min_{W} \|WR - \Xi\|_F^2 + \lambda\|W\|_F^2, \tag{A3}$$

where $0 < \lambda \ll 1$ is a regularization parameter and $\|\cdot\|_F$ denotes the Frobenius norm. The corresponding solution to the regularized least-squares problem is given by

$$W_{OUT} = \Xi R^T(RR^T + \lambda I)^{-1}. \tag{A4}$$

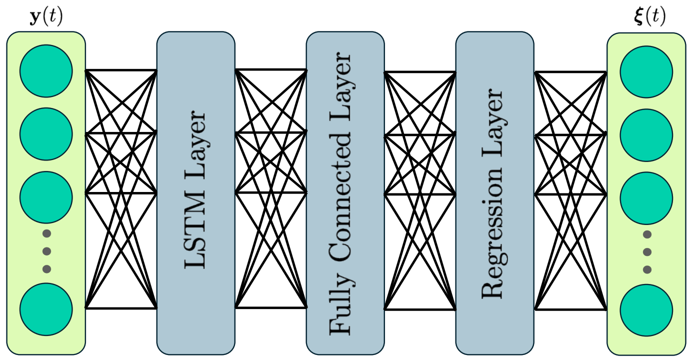

Figure A2: Architecture of the Long Short Term Memory (LSTM) model. The LSTM network architecture processes input data $\mathbf{y}(t)$ through an LSTM network followed by a fully connected layer and a regression layer to predict the output vector $\boldsymbol{\xi}(t)$.

Training a RC network is computationally inexpensive (see Table 2) because only the output layer W_{OUT} needs to be learned and its optimal value is known in closed form (A4).

To determine the optimal architecture for this study, we performed a hyperparameter search over several key parameters. The search space includes the learning rate $\alpha \in \{0.2, 0.4, 0.8, 1.0\}$, the reservoir size $N_r \in \{100, 250, 500, 1000\}$, and the sparsity of the reservoir weight matrix W_R, $s \in \{10\%, 20\%, 40\%\}$. The final hyperparameters that we use for this paper, determined by the smallest reconstruction relative error on the training data, are detailed in Table 1.

A2 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are a more complex and powerful type of RNN designed specifically to overcome challenges with learning long-term dependencies in data (Hochreiter & Schmidhuber, 1997; Van Houdt et al., 2020). Figure A2 depicts the architecture we use to learn the kernel coordinates $\boldsymbol{\xi}(t)$.

The network processes the input vector of in situ observations $\mathbf{y}(t)$. This input is first fed into an LSTM layer composed of memory cells that use gating mechanisms (input, forget, and output gates) to regulate the flow of information. The hidden state from the LSTM layer is then passed to a fully connected layer, which learns nonlinear combinations of the features extracted by the LSTM. Subsequently, a regression layer maps these high-level features to the target values, here the optimal kernel coordinates $\hat{\boldsymbol{\xi}}(t)$.

To optimize the predictive performance of the LSTM network, we conducted hyperparameter tuning. The parameters investigated included the number of hidden units $h_t \in \{100, 200, 300\}$, the initial learning rates $\alpha \in \{10^{-2}, 5 \times 10^{-3}, 10^{-3}\}$, the learning rate drop period after $\tau \in \{50, 100\}$ epochs, and the corresponding learning rate drop factors $\gamma \in \{0.1, 0.5\}$. The optimal hyperparameters were chosen based on the combination of hyperparameters that yielded the lowest reconstruction error on the training data. These parameters are reported in Table 1.

Appendix B Autoencoder Experiments

S-DEIM uses POD modes for dimensionality reduction. Since this linear method led to relatively large reconstruction errors during seasonal changes in May and November (see the best fits in Figure 4), we also considered autoencoders as a nonlinear method

Layer	Type	Window	Stride
1	Conv	3×3	1
2	Pooling (Max)	2×2	2
3	Conv	3×3	1
4	Pooling (Max)	2×2	2
5	Conv	3×3	1
6	Pooling (Max)	3×3	3
7	Conv	3×3	1
8	Pooling (Max)	3×3	3
9	TransConv	3×3	3
10	TransConv	3×3	3
11	TransConv	2×2	2
12	TransConv	2×2	2

Table B1: Architecture of the CAE. Conv refer to a 2D convolution layer and TransConv refers to a transposed convolution layer.

of dimensionality reduction. In particular, we used a Convolutional Autoencoder (CAE), a deep neural network trained to produce an identity mapping by compressing the dimension of its input into the latent space (encoding) and then recovering the original input by constructing an inverse mapping from the latent space to the output (decoding) (Masci et al., 2011; Vincent et al., 2008).

Unfortunately, these efforts did not reduce the reconstruction error. In fact, the CAE results were similar to the Q-DEIM reconstruction error which is roughly 40% larger than S-DEIM reconstruction error. Nevertheless, for completeness, we outline the CAE architecture used in our experiments in Table B1. The encoder comprises four convolutional layers, each followed with a max pooling layer. The decoder has four transposed convolutional layers. This architecture is similar to the one used in Barth et al. (2020). In our experiments, we did tinker with the window size and the stride of each layer, however, no substantial improvements were observed.

References

Argaud, J. P., Bouriquet, B., Gong, H., Maday, Y., & Mula, O. (2017). Stabilization of (G)EIM in presence of measurement noise: Application to nuclear reactor physics. In M. L. Bittencourt, N. A. Dumont, & J. S. Hesthaven (Eds.), *Spectral and high order methods for partial differential equations icosahom 2016* (pp. 133–145). Cham: Springer International Publishing.

Banzon, V., Smith, T. M., Chin, T. M., Liu, C., & Hankins, W. (2016). A long-term record of blended satellite and in situ sea-surface temperature for climate monitoring, modeling and environmental studies. *Earth System Science Data*, *8*(1), 165–176. doi: 10.5194/essd-8-165-2016

Barrault, M., Maday, Y., Nguyen, N. G., & Patera, A. T. (2004). An 'empirical interpolation' method: application to efficient reduced-basis discretization of partial differential equations. *Comptes Rendus Mathematique*, *339*(9), 667-672. doi: https://doi.org/10.1016/j.crma.2004.08.006

Barth, A., Alvera-Azcárate, A., Licer, M., & Beckers, J.-M. (2020). DINCAE 1.0: a convolutional neural network with error estimates to reconstruct sea surface temperature satellite observations. *Geoscientific Model Development*, *13*(3), 1609–1622. doi: 10.5194/gmd-13-1609-2020

Barth, A., Alvera-Azcárate, A., Troupin, C., & Beckers, J.-M. (2022). DINCAE 2.0:

multivariate convolutional neural network with error estimates to reconstruct sea surface temperature satellite and altimetry observations. *Geoscientific Model Development*, *15*(5), 2183–2196. doi: 10.5194/gmd-15-2183-2022

Beckers, J. M., & Rixen, M. (2003). EOF calculations and data filling from incomplete oceanographic datasets. *Journal of Atmospheric and Oceanic Technology*, *20*(12), 1839 - 1856. doi: 10.1175/1520-0426(2003)020⟨1839:ECADFF⟩2.0.CO;2

Berkooz, G., Holmes, P., & Lumley, J. L. (1993). The proper orthogonal decomposition in the analysis of turbulent flows. *Annual review of fluid mechanics*, *25*(1), 539–575.

Bi, K., Xie, L., Zhang, H., Chen, X., Gu, X., & Tian, Q. (2023). Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, *619*(7970), 533–538. Retrieved from http://dx.doi.org/10.1038/s41586-023-06185-3 doi: 10.1038/s41586-023-06185-3

Bodnar, C., Bruinsma, W. P., Lucic, A., Stanley, M., Allen, A., Brandstetter, J., … Perdikaris, P. (2025, May). A foundation model for the earth system. *Nature*, *641*(8065), 1180–1187. doi: 10.1038/s41586-025-09005-y

Bottou, L., & Bousquet, O. (2007). The tradeoffs of large scale learning. In J. Platt, D. Koller, Y. Singer, & S. Roweis (Eds.), *Advances in neural information processing systems* (Vol. 20). Curran Associates, Inc. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2007/file/0d3180d672e08b4c5312dcdafdf6ef36-Paper.pdf

Callaham, J. L., Maeda, K., & Brunton, S. L. (2019). Robust flow reconstruction from limited measurements via sparse representation. *Phys. Rev. Fluids*, *4*, 103907. doi: 10.1103/PhysRevFluids.4.103907

Chaturantabut, S., & Sorensen, D. C. (2010). Nonlinear model reduction via discrete empirical interpolation. *SIAM Journal on Scientific Computing*, *32*(5), 2737-2764. doi: 10.1137/090766498

Chelton, D. B., & Wentz, F. J. (2005). Global microwave satellite observations of sea surface temperature for numerical weather prediction and climate research. *Bulletin of the American Meteorological Society*, *86*(8), 1097–1116. doi: 10.1175/BAMS-86-8-1097

Clark, E., Brunton, S. L., & Kutz, J. N. (2021). Multi-fidelity sensor selection: Greedy algorithms to place cheap and expensive sensors with cost constraints. *IEEE Sensors Journal*, *21*(1), 600-611. doi: 10.1109/JSEN.2020.3013094

Courtier, P., Andersson, E., Heckley, W., Vasiljevic, D., Hamrud, M., Hollingsworth, A., … Pailleux, J. (1998). The ECMWF implementation of three-dimensional variational assimilation (3D-VAR). I: Formulation. *Quarterly Journal of the Royal Meteorological Society*, *124*(550), 1783-1807. doi: 10.1002/qj.49712455002

Courtier, P., Thépaut, J.-N., & Hollingsworth, A. (1994). A strategy for operational implementation of 4D-Var, using an incremental approach. *Quarterly Journal of the Royal Meteorological Society*, *120*(519), 1367-1387. doi: 10.1002/qj.49712051912

Cummings, J. A., & Smedstad, O. M. (2013). Variational data assimilation for the global ocean. In *Data assimilation for atmospheric, oceanic and hydrologic applications (vol. ii)* (pp. 303–343). Springer.

Deser, C., Alexander, M. A., Xie, S.-P., & Phillips, A. S. (2010). Sea surface temperature variability: Patterns and mechanisms. *Annual Review of Marine Science*, *2*(Volume 2, 2010), 115-143. doi: 10.1146/annurev-marine-120408-151453

Dickey, T., Lewis, M., & Chang, G. (2006). Optical oceanography: Recent advances and future directions using global remote sensing and in situ observations. *Reviews of Geophysics*, *44*(1). doi: 10.1029/2003RG000148

Drmač, Z., & Gugercin, S. (2016). A new selection operator for the discrete empirical interpolation method—Improved a priori error bound and extensions. *SIAM Journal on Scientific Computing*, *38*(2), A631-A648. doi: 10.1137/15M1019271

Evensen, G. (2003). The ensemble kalman filter: theoretical formulation and practical implementation. *Ocean Dynamics*, *53*(4), 343–367. doi: 10.1007/s10236-003-0036-9

Everson, R., & Sirovich, L. (1995, Aug). Karhunen–loève procedure for gappy data. *J. Opt. Soc. Am. A*, *12*(8), 1657–1664. doi: 10.1364/JOSAA.12.001657

Farazmand, M. (2024). Sparse discrete empirical interpolation method: State estimation from few sensors. *SIAM Journal on Scientific Computing*, *46*(6), A3658-A3680. doi: 10.1137/24M1636344

Farazmand, M. (2026). State estimation using sparse DEIM and recurrent neural networks. *Proc. R. Soc. A (in press)*. Retrieved from https://arxiv.org/abs/2410.15982 doi: 10.48550/arXiv.2410.15982

Goddard, L., Mason, S., Zebiak, S., Ropelewski, C., Basher, R., & Cane, M. (2001). Current approaches to seasonal to interannual climate predictions. *International Journal of Climatology*, *21*(9), 1111-1152. doi: https://doi.org/10.1002/joc.636

Goh, E., Yepremyan, A., Wang, J., & Wilson, B. (2024). MAESSTRO: Masked autoencoders for sea surface temperature reconstruction under occlusion. *Ocean Science*, *20*(5), 1309–1323. Retrieved from https://os.copernicus.org/articles/20/1309/2024/ doi: 10.5194/os-20-1309-2024

Gorthi, S. S., Beyou, S., & Mémin, E. (2011). Analysis of SST images by weighted ensemble transform Kalman filter. In *2011 ieee international geoscience and remote sensing symposium* (p. 4172-4175). doi: 10.1109/IGARSS.2011.6050049

Gu, M., & Eisenstat, S. C. (1996). Efficient algorithms for computing a strong rank-revealing qr factorization. *SIAM Journal on Scientific Computing*, *17*(4), 848–869.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735-1780. doi: 10.1162/neco.1997.9.8.1735

Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, *24*(6), 417-441. doi: 10.1037/h0071325

Kakasenko, L., Alexanderian, A., Farazmand, M., & Saibaba, A. K. (2025). *Bridging the gap between deterministic and probabilistic approaches to state estimation.* doi: 10.48550/arXiv.2505.04004

Karnik, N., Bhangale, Y., Abdo, M. G., Klishin, A. A., Cogliati, J. J., Brunton, B. W., … Manohar, K. (2025). PySensors 2.0: A Python package for sparse sensor placement. *arXiv 2509.08017*. Retrieved from https://arxiv.org/abs/2509.08017

Kennedy, J. J. (2013, November). A review of uncertainty in in situ measurements and data sets of sea surface temperature. *Reviews of Geophysics*, *52*, 1-32. Retrieved from https://agupubs.onlinelibrary.wiley.com/doi/full/10.1002/2013RG000434 doi: https://doi.org/10.1002/2013RG000434

Kent, E. C., Kennedy, J. J., Berry, D. I., & Smith, R. O. (2010, September/October). Effects of instrumentation changes on sea surface temperature measured in situ. *WIREs Climate Change*, *1*(5), 718-728.

Klishin, A. A., Kutz, J. N., & Manohar, K. (2023). Data-induced interactions of sparse sensors, arxiv 2307.11838.
doi: 10.48550/arXiv.2307.11838

Koch, O., & Lubich, C. (2007). Dynamical low-rank approximation. *SIAM Journal on Matrix Analysis and Applications*, *29*(2), 434-454. doi: 10.1137/050639703

Koner, P. K., Harris, A., & Maturi, E. (2016). Hybrid cloud and error masking to improve the quality of deterministic satellite sea surface temperature retrieval and data coverage. *Remote Sensing of Environment*, *174*, 266–278.

Lobashev, A. A., Turko, N. A., Ushakov, K. V., Kaurkin, M. N., & Ibrayev, R. A. (2023). Concrete autoencoder for the reconstruction of sea temperature field from sparse measurements. *Journal of Marine Science and Engineering*, *11*(2). doi: 10.3390/jmse11020404

Lukosevicius, M., & Jaeger, H. (2009). Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, *3*(3), 127-149. doi: 10.1016/j.cosrev.2009.03.005

Lumey, J. L. (1975). *Stochastic tools in turbulence.* New York: Elsevier.

Manohar, K., Brunton, B. W., Kutz, J. N., & Brunton, S. L. (2018). Data-driven sparse sensor placement for reconstruction: Demonstrating the benefits of exploiting known patterns. *IEEE Control Systems Magazine*, *38*(3), 63-86. doi: 10.1109/MCS.2018.2810460

Masci, J., Meier, U., Cireşan, D., & Schmidhuber, J. (2011). Stacked convolutional auto-encoders for hierarchical feature extraction. In *Artificial neural networks and machine learning – icann 2011* (p. 52–59). Springer Berlin Heidelberg. doi: 10.1007/978-3-642-21735-7_7

McCall, J. (1998). Advances in National Data Buoy Center technology. In *Ieee oceanic engineering society. oceans'98. conference proceedings (cat. no.98ch36259)* (Vol. 1, p. 544-548). doi: 10.1109/OCEANS.1998.725806

Minnett, P., Alvera-Azcárate, A., Chin, T., Corlett, G., Gentemann, C., Karagali, I., ... Vazquez-Cuervo, J. (2019). Half a century of satellite remote sensing of sea-surface temperature. *Remote Sensing of Environment*, *233*, 111366. Retrieved from https://www.sciencedirect.com/science/article/pii/S0034425719303852 doi: https://doi.org/10.1016/j.rse.2019.111366

Pathak, J., Subramanian, S., Harrington, P., Raja, S., Chattopadhyay, A., Mardani, M., ... Anandkumar, A. (2022). FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators. *arXiv preprint arXiv:2202.11214*.

Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, *2*(11), 559–572. doi: 10.1080/14786440109462720

Peherstorfer, B., Drmač, Z., & Gugercin, S. (2020). Stability of discrete empirical interpolation and gappy proper orthogonal decomposition with randomized and deterministic sampling points. *SIAM Journal on Scientific Computing*, *42*(5), A2837-A2864. doi: 10.1137/19M1307391

Reynolds, R. W., Rayner, N. A., Smith, T. M., Stokes, D. C., & Wang, W. (2002). An improved in situ and satellite sst analysis for climate. *Journal of Climate*, *15*(13), 1609–1625. doi: 10.1175/1520-0442(2002)015⟨1609:AIISAS⟩2.0.CO;2

Reynolds, R. W., & Smith, T. M. (1994, June). Improved global sea surface temperature analyses using optimum interpolation. *Journal of Climate*, *7*(6), 929-948.

Reynolds, R. W., Smith, T. M., Liu, C., Chelton, D. B., Casey, K. S., & Schlax, M. G. (2007, November). Daily high-resolution-blended analyses for sea surface temperature. *Journal of Climate*, *20*(22), 5473–5496.

Rozier, D., Birol, F., Cosme, E., Brasseur, P., Brankart, J. M., & Verron, J. (2007). A reduced-order Kalman filter for data assimilation in physical oceanography. *SIAM Review*, *49*(3), 449-465. doi: 10.1137/050635717

Saito, Y., Nonomura, T., Yamada, K., Nakai, K., Nagata, T., Asai, K., ... Tsubakino, D. (2021). Determinant-based fast greedy sensor selection algorithm. *IEEE Access*, *9*, 68535-68551. doi: 10.1109/ACCESS.2021.3076186

Sirovich, L. (1987). Turbulence and the dynamics of coherent structures. Part I: Coherent structures. *Quarterly of Applied Mathematics*, *45*(3), 561–571. doi: 10.1090/qam/910462

Smith, T. M., Reynolds, R. W., Livezey, R. E., & Stokes, D. C. (1996). Reconstruction of historical sea surface temperatures using empirical orthogonal functions. *J. Climate*, *9*, 1403–1420.

Taylor, J., & Feng, M. (2022). A deep learning model for forecasting global monthly mean sea surface temperature anomalies. *Frontiers in Climate*, *4*. doi: 10.3389/fclim.2022.932932

Van Houdt, G., Mosquera, C., & Nápoles, G. (2020). A review on the long short-term memory model. *Artificial Intelligence Review*, *53*(8), 5929-5955. doi: 10.1007/s10462-020-09838-1

Venegas, R. M., Acevedo, J., & Treml, E. A. (2023). Three decades of ocean warming impacts on marine ecosystems: A review and perspective. *Deep Sea Research Part II: Topical Studies in Oceanography*, *212*, 105318. doi: 10.1016/j.dsr2.2023.105318

Vincent, P., Larochelle, H., Bengio, Y., & Manzagol, P.-A. (2008). Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on machine learning* (p. 1096–1103). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/1390156.1390294 doi: 10.1145/1390156.1390294

Welper, G., & Keene, B. (2025). Approximation, estimation and optimization errors for a deep neural network. *Transactions on Machine Learning Research*. Retrieved from https://openreview.net/forum?id=dzND5haNvA

Xu, S., Dai, D., Cui, X., Yin, X., Jiang, S., Pan, H., & Wang, G. (2023). A deep learning approach to predict sea surface temperature based on multiple modes. *Ocean Modelling*, *181*, 102158. doi: 10.1016/j.ocemod.2022.102158